

Expt 6a: First come first serve:

[Back To Practice](#)

Completed

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main() {
3     int n;
4     printf("Enter number of processes: \n");
5     scanf("%d", &n);
6     int at[n], bt[n], ct[n], tat[n], wt[n];
7     for (int i = 0; i < n; i++) {
8         printf("Enter Arrival Time and Burst Time for P%d: \n", i + 1);
9         scanf("%d %d", &at[i], &bt[i]);
10    }
11    ct[0] = at[0] + bt[0];
12    for (int i = 1; i < n; i++) {
13        if (ct[i-1] < at[i])
14            ct[i] = at[i] + bt[i];
15        else
16            ct[i] = ct[i-1] + bt[i];
17    }
```

[Test Cases 2](#)[Run Sample Cases](#)[Run All Cases](#)

Custom Test Case

> Custom Test

Sample Test Case

[Back To Practice](#)

Sample Input 1

```
3
0 2
1 2
2 3
```

Sample Output 1

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33

Average Waiting Time: 1.00

Select Language: C (GCC 9.2.0)

```
12     for (int i = 1; i < n; i++) {
13         if (ct[i-1] < at[i])
14             ct[i] = at[i] + bt[i];
15         else
16             ct[i] = ct[i-1] + bt[i];
17     }
18     float totalTAT = 0, totalWT = 0;
19     for (int i = 0; i < n; i++) {
20         tat[i] = ct[i] - at[i];
21         wt[i] = tat[i] - bt[i];
22         totalTAT += tat[i];
23         totalWT += wt[i];
24     }
25     printf("\nAverage Turnaround Time: %.2f\n", totalTAT / n);
26     printf("Average Waiting Time: %.2f\n", totalWT / n);
27     return 0;
28 }
```

[Test Cases 2](#)[Run Sample Cases](#)[Run All Cases](#)

Custom Test Case

> Custom Test

Sample Test Case

[Back To Practice](#)

Q1

Save

Sample Input 2

```

3
0 5
5 3
8 2

```

Sample Output 2

```

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 0.00

```

Custom Test Case

Custom Test

Success

Input :

```

3
0 2
1 2
2 3

```

Expected Output :

```

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00

```

Output :

```

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00

```

Expt 6b: Non-Preemptive shortest job first:

[Back To Practice](#)

Q1

Save

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)

Completed

Select Language: C (GCC 9.2.0)

```

1 #include<stdio.h>
2 int main(){
3     int n;
4     printf("Enter number of processes: \n");
5     scanf("%d",&n);
6     int at[n],bt[n],ct[n],tat[n],wt[n],done[n];
7     for (int i = 0; i < n; i++){
8         printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
9         scanf("%d %d",&at[i],&bt[i]);
10        done[i] = 0;
11    }
12    int current_time=0,completed=0;
13    while(completed<n){
14        int idx=-1;
15        int min_bt=99999;
16        for(int i=0;i<n;i++){
17            if(!done[i] && at[i]<=current_time){

```

Test Cases 2

Run Sample Cases Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

[Back To Practice](#)

<

Q1

>

Save

Sample Input 1

4
0 6
0 8
0 7
0 3

Sample Output 1

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00

Select Language: C (GCC 9.2.0)

```

29     }
30     current_time+=bt[idx];
31     ct[idx]=current_time;
32     tat[idx]=ct[idx]-at[idx];
33     wt[idx]=tat[idx]-bt[idx];
34     done[idx]=1;
35     completed++;
36     }
37     float totalTAT=0,totalWT=0;
38     for(int i=0;i<n;i++){
39         totalTAT+=tat[i];
40         totalWT+=wt[i];
41     }
42     printf("\nAverage Turnaround Time = %.2f\n",totalTAT/n);
43     printf("Average Waiting Time = %.2f\n",totalWT/n);
44     return 0;
45 }

```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Input 2

3
0 3
5 2
5 1

Sample Output 2

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time = 2.33
Average Waiting Time = 0.33

Custom Test

Success

Input :

4
0 6
0 8
0 7
0 3

Expected Output :

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00

Output :

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00

Expt 6c: Non-Preemptive priority scheduling:

[Back To Practice](#)

Completed

Non-Preemptive Priority Scheduling algorithm.

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

Q1

Save

Select Language: C (GCC 9.2.0)

```
1 #include<stdio.h>
2 int main(){
3     int n;
4     printf("Enter number of processes: \n");
5     scanf("%d",&n);
6     int at[n],bt[n],pr[n],ct[n],tat[n],wt[n],done[n];
7     for(int i=0;i<n;i++){
8         printf("Enter AT, BT and Priority for P%d: \n",i+1);
9         scanf("%d %d %d",&at[i],&bt[i],&pr[i]);
10        done[i]=0;
11    }
12    int current_time=0,completed=0;
13    while(completed<n){
14        int idx=-1;
15        int min_pr=99999;
16        for(int i=0;i<n;i++){
17            if(!done[i] && at[i] <= current_time){
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

[Back To Practice](#)

Example 1

Sample Input 1

```
3
0 5 2
1 3 1
2 2 3
```

Sample Output 1

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

Q1

Save

Select Language: C (GCC 9.2.0)

```
30     current_time+=bt[idx];
31     ct[idx]=current_time;
32     tat[idx]=ct[idx]-at[idx];
33     wt[idx]=tat[idx]-bt[idx];
34     done[idx]=1;
35     completed++;
36 }
37 float totalTAT=0,totalWT=0;
38 for(int i=0;i<n;i++){
39     totalTAT+=tat[i];
40     totalWT+=wt[i];
41 }
42 printf("\nAverage Turnaround Time = %.2f\n",totalTAT/n);
43 printf("Average Waiting Time = %.2f\n",totalWT/n);
44
45 return 0;
46 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

Back To Practice
< Q1 >
Save

4

0 4 3

0 3 1

2 2 4

5 1 2

Sample Output 2

Enter number of processes:

Enter AT, BT and Priority for P1:

Enter AT, BT and Priority for P2:

Enter AT, BT and Priority for P3:

Enter AT, BT and Priority for P4:

Average Turnaround Time = 5.25

Average Waiting Time = 2.75

Custom test Case

Input :

```
3
0 5 2
1 3 1
2 2 3
```

Expected Output :

Enter number of processes:

Enter AT, BT and Priority for P1:

Enter AT, BT and Priority for P2:

Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67

Average Waiting Time = 3.33

Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

Expt 6d: Round robin

Back To Practice
< Q1 >
Save

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the Time Quantum.
- The next lines contain two integers for each process, where

- The first integer denotes the Arrival Time (AT)

Select Language: C (GCC 9.2.0)

```
1 #include<stdio.h>
2 int main(){
3     int n,tq;
4     printf("Enter number of processes: \n");
5     scanf("%d",&n);
6     printf("Enter Time Quantum: \n");
7     scanf("%d",&tq);
8     int at[n],bt[n],rem[n],ct[n];
9     for(int i=0;i<n;i++){
10        printf("Enter AT and BT for P%d: \n",i+1);
11        scanf("%d %d",&at[i],&bt[i]);
12        rem[i]=bt[i];
13    }
14    int time =0,done=0;
15    int tat[n],wt[n];
16    while(done<n){
17        int progressed=0;
```

Test Cases 2

Run Sample Cases Run All Cases

Custom Test Case

> Custom Test

Sample Input 1

```

3
2
0 5
1 3
2 1

```

Sample Output 1

Enter number of processes:

Enter Time Quantum:

Enter AT and BT for P1:

Enter AT and BT for P2:

Enter AT and BT for P3:

Average Turnaround Time: 6.33

Select Language: C (GCC 9.2.0)

```

30     }
31     }
32     }
33     if(!progressed)
34         time++;
35     }
36     float totalTAT=0,totalWT=0;
37     for (int i=0;i<n;i++){
38         tat[i]=ct[i]-at[i];
39         wt[i]=tat[i]-bt[i];
40         totalTAT+=tat[i];
41         totalWT+=wt[i];
42     }
43     printf("\nAverage Turnaround Time: %.2f\n",totalTAT/n);
44     printf("Average Waiting Time: %.2f\n",totalWT/n);
45     return 0;
46 }

```

Test Cases 2

Run Sample Cases
Run All Cases

Custom Test Case

> Custom Test

Back To Practice
Q1
Save

Sample Input 1

```

2
0 5
1 3
2 1

```

Sample Output 1

Enter number of processes:

Enter Time Quantum:

Enter AT and BT for P1:

Enter AT and BT for P2:

Enter AT and BT for P3:

Average Turnaround Time: 6.33

Average Waiting Time: 3.33

Input :

```

3
2
0 5
1 3
2 1

```

Expected Output :

```

Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:

Average Turnaround Time: 6.33
Average Waiting Time: 3.33

```

Output :

```

Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:

Average Turnaround Time: 6.33
Average Waiting Time: 3.33

```